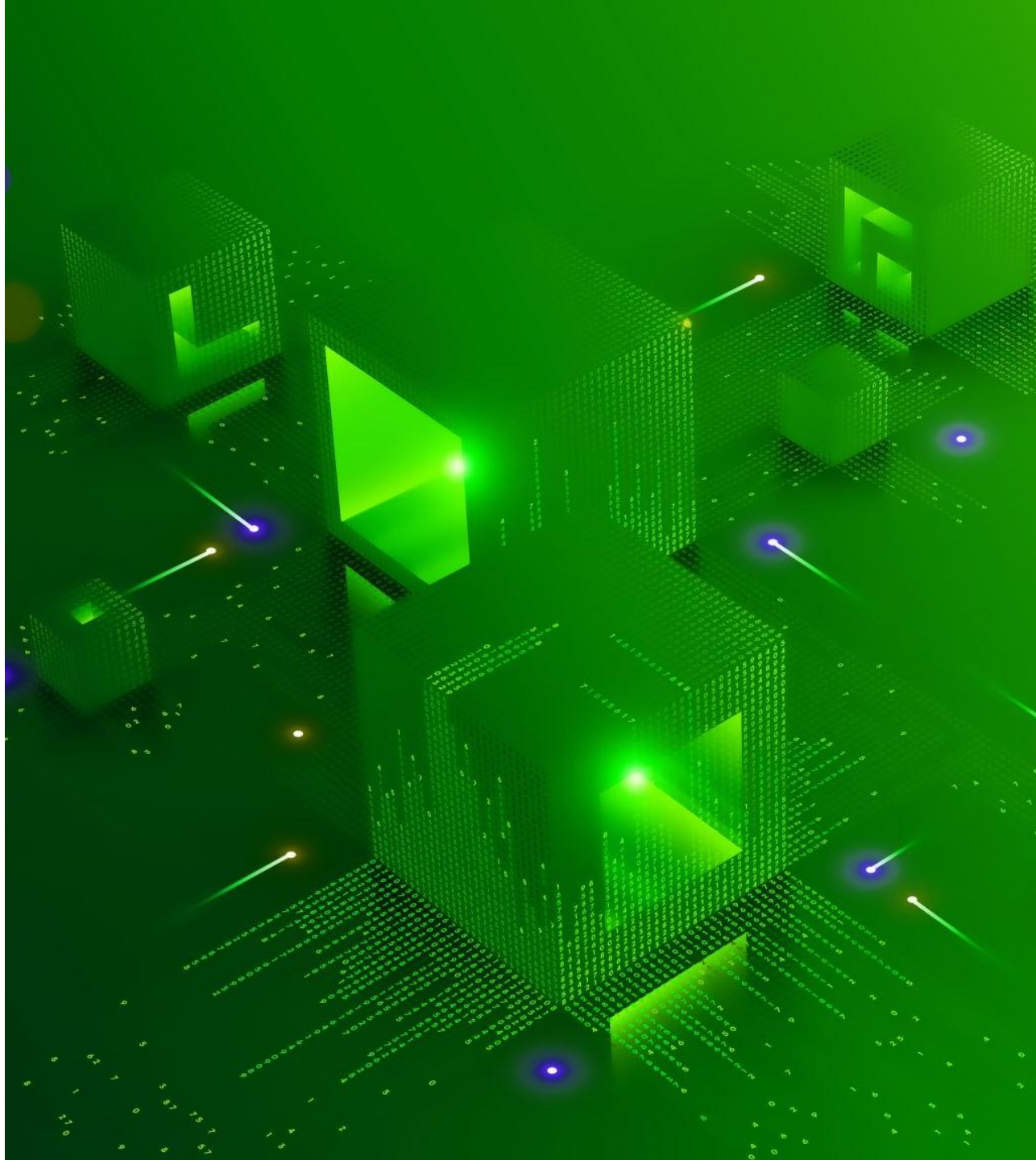
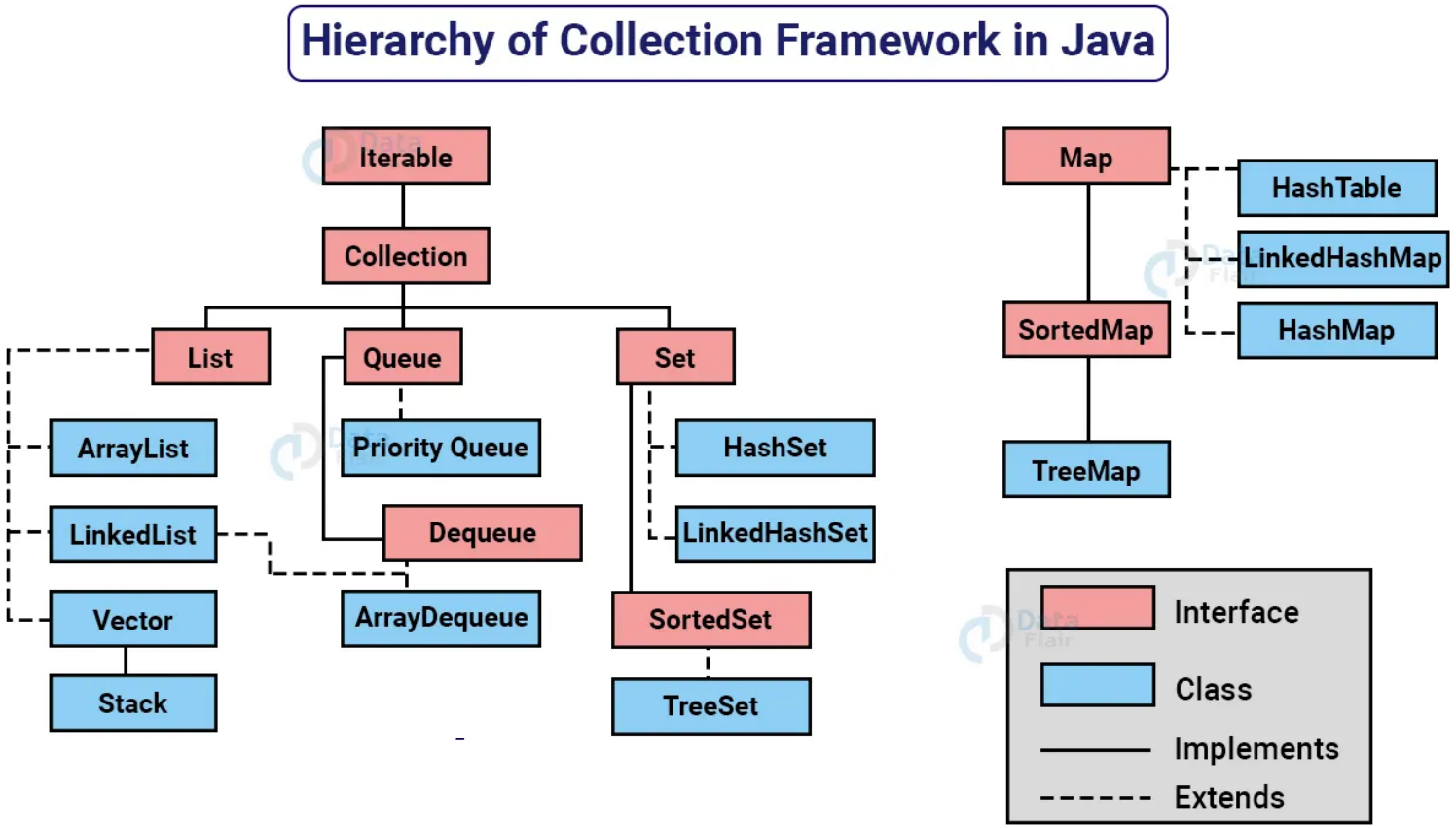
Java Collections and Generics

Introduction

- Java Collections Framework (JCF) provides reusable data structures
- Generics allow type safety and code reusability
- Improves efficiency, scalability, and maintainability



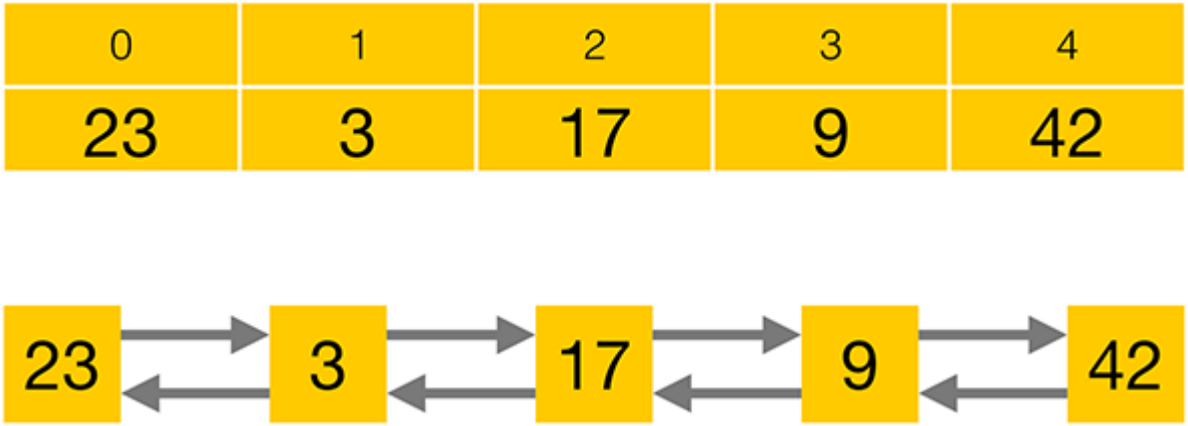


Java Collections Framework Overview

Collections framework defines interfaces and implementation

Oracle : A collection is an object that represents a group of objects (such as the classic Vector class).

ArrayList vs LinkedList



ArrayList

Create an `ArrayList` object called **cars** that will store strings:

```
import java.util.ArrayList; // Import the ArrayList class

ArrayList<String> cars = new ArrayList<String>(); // Create an ArrayList object
```

Now you can use methods like `add()`, `get()`, `set()`, and `remove()` to manage your list of elements.

ArrayList...

To add elements to an `ArrayList`, use the `add()` method:

Example

```
import java.util.ArrayList;

public class Main {
  public static void main(String[] args) {
    ArrayList<String> cars = new ArrayList<String>();
    cars.add("Volvo");
    cars.add("BMW");
    cars.add("Ford");
    cars.add("Mazda");
    System.out.println(cars);
  }
}
```

ArrayList...

You can also add an element at a specified position by referring to the index number:

Example

```
import java.util.ArrayList;

public class Main {
  public static void main(String[] args) {
    ArrayList<String> cars = new ArrayList<String>();
    cars.add("Volvo");
    cars.add("BMW");
    cars.add("Ford");

    cars.add(0, "Mazda"); // Insert element at the beginning of the list (0)

    System.out.println(cars);
  }
}
```

ArrayList...

To access an element in the `ArrayList`, use the `get()` method and refer to the index number:

Example

```
cars.get(0);  // Get the first element
```

ArrayList...

To modify an element, use the `set()` method and refer to the index number:

Example

```
cars.set(0, "Opel");
```

ArrayList...

To remove an element, use the `remove()` method and refer to the index number:

Example

```
cars.remove(0);
```

ArrayList...

To remove all the elements in the `ArrayList`, use the `clear()` method:

Example

```
cars.clear();
```

ArrayList...

To find out how many elements an `ArrayList` have, use the `size` method:

Example

```
cars.size();
```

ArrayList...

```
public class Main {
  public static void main(String[] args) {
    ArrayList<String> cars = new ArrayList<String>();
    cars.add("Volvo");
    cars.add("BMW");
    cars.add("Ford");
    cars.add("Mazda");
    for (int i = 0; i < cars.size(); i++) {
      System.out.println(cars.get(i));
    }
  }
}
```

ArrayList...

You can also loop through an `ArrayList` with the **for-each** loop:

Example

```
public class Main {
  public static void main(String[] args) {
    ArrayList<String> cars = new ArrayList<String>();
    cars.add("Volvo");
    cars.add("BMW");
    cars.add("Ford");
    cars.add("Mazda");
    for (String i : cars) {
      System.out.println(i);
    }
  }
}
```

ArrayList...

Create an `ArrayList` to store numbers (add elements of type `Integer`):

```
import java.util.ArrayList;

public class Main {
  public static void main(String[] args) {
    ArrayList<Integer> myNumbers = new ArrayList<Integer>();
    myNumbers.add(10);
    myNumbers.add(15);
    myNumbers.add(20);
    myNumbers.add(25);
    for (int i : myNumbers) {
      System.out.println(i);
    }
  }
}
```

ArrayList...

Sort an ArrayList of Strings:

```
import java.util.ArrayList;
import java.util.Collections;  // Import the Collections class

public class Main {
  public static void main(String[] args) {
    ArrayList<String> cars = new ArrayList<String>();
    cars.add("Volvo");
    cars.add("BMW");
    cars.add("Ford");
    cars.add("Mazda");
    Collections.sort(cars);  // Sort cars
    for (String i : cars) {
      System.out.println(i);
    }
  }
}
```

Generics

Java Generics

Generics allow you to write classes, interfaces, and methods that work with different data types, without having to specify the exact type in advance.

This makes your code more flexible, reusable, and type-safe.

Why Use Generics?

- **Code Reusability:** Write one class or method that works with different data types.
- **Type Safety:** Catch type errors at compile time instead of runtime.
- **Cleaner Code:** No need for casting when retrieving objects.

Generics...

```java
class Box<T> {
  T value; // T is a placeholder for any data type

  void set(T value) {
    this.value = value;
  }

  T get() {
    return value;
  }
}

public class Main {
  public static void main(String[] args) {
    // Create a Box to hold a String
    Box<String> stringBox = new Box<>();
    stringBox.set("Hello");
    System.out.println("Value: " + stringBox.get());

    // Create a Box to hold an Integer
    Box<Integer> intBox = new Box<>();
    intBox.set(50);
    System.out.println("Value: " + intBox.get());
  }
}
```

Generics...

You can also create methods that work with any data type using generics:

```java
public class Main {
  // Generic method: works with any type T
  public static <T> void printArray(T[] array) {
    for (T item : array) {
      System.out.println(item);
    }
  }

  public static void main(String[] args) {
    // Array of Strings
    String[] names = {"Jenny", "Liam"};

    // Array of Integers
    Integer[] numbers = {1, 2, 3};

    // Call the generic method with both arrays
    printArray(names);
    printArray(numbers);
  }
}
```

Generics

Before Java v5

```
public class Box {

    private Object object;



    public void set(Object object) { this.object =
object; }

    public Object get() { return object; }
```



```
}
```

After Java v5

```
/**

 * Generic version of the Box class.

 * @param <T> the type of the value being boxed

 */

public class Box<T> {

    // T stands for "Type"

    private T t;


    public void set(T t) { this.t = t; }

    public T get() { return t; }

}
```

Generics

```
Box<Integer> integerBox;
Box<Integer> integerBox = new Box<Integer>();
```

- all occurrences of Object are replaced by T.
- A type variable can be any non-primitive type : any class type, any interface type, any array type, or even another type variable.

  - E - Element (used extensively by the Java Collections Framework)
  - K - Key
  - N - Number
  - T - Type
  - V - Value
  - S,U,V etc. - 2nd, 3rd, 4th types

Generics (Multiple types)

```
public interface Pair<K, V> {
    public K getKey();
    public V getValue();
}
```

```
public class OrderedPair<K, V> implements Pair<K, V> {

    private K key;
    private V value;

    public OrderedPair(K key, V value) {
            this.key = key;
            this.value = value;
    }

    public K getKey()          { return key; }
    public V getValue() { return value; }
}
```

Generics : Why ?

```
// in Java 1.4:
ArrayList names = new ArrayList();
names.add("XYX");
names.add("XXX");
String teacher = (String) names.get(0);

// this will compile but crash at runtime; bad
Point oops = (Point) names.get(1);
```

Wildcards

- `?` indicates a *wild-card* type parameter, one that can be any type.
  - `List<?> list = new List<?>();  // anything`
- Difference between `List<?>` and `List<Object>`:
  - `?` can become any particular type; `Object` is just one such type.
  - `List<Object>` is restrictive; wouldn't take a `List<String>`
- Difference btwn. `List<Foo>` and `List<? extends Foo>`:
  - The latter binds to a particular `Foo` subtype and allows ONLY that.
    - e.g. List<? extends Animal> might store only Giraffes but not Zebras
  - The former allows anything that is a subtype of `Foo` in the same list.
    - e.g. List<Animal> could store both Giraffes and Zebras